Task 1: Custom IAM Policy for EC2 Restriction:

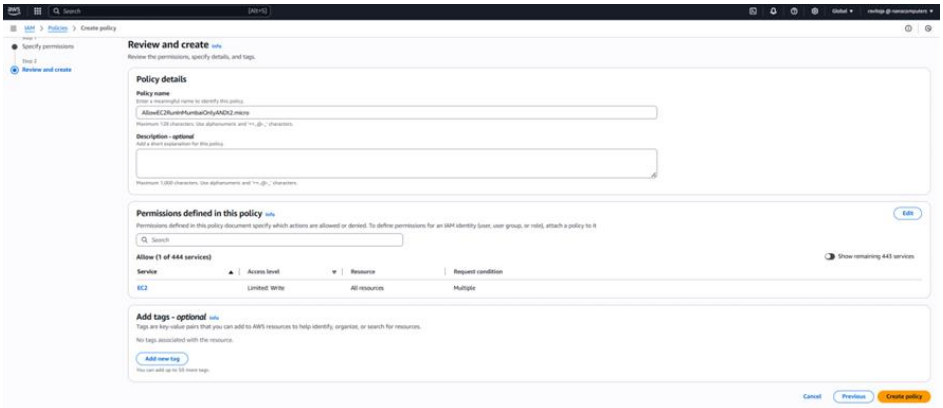
Create a custom IAM policy that allows users to: Only launch T2.micro EC2 instances (no other instance types).

Restrict deployment to the Mumbai region (ap-south-1).

Tools to Use:

AWS Policy Generator (for JSON template), IAM Console (to attach the policy).

```json
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "ec2:RunInstances",
            "Resource": "*",
            "Condition": {
                "StringEquals": {
                    "ec2:InstanceType": "t2.micro",
                    "aws:RequestedRegion": "ap-south-1"
                }
            }
        }
    ]
}
```



Task 2: AWS CLI IAM Operations

Perform the following via AWS CLI: Create an IAM user, Create a user group, Add users to the group. Attach a custom policy (e.g., the one created in Task 1).

* Example Commands:

bash script

Create user

aws iam create-user --user-name DevOps_Intern

Create group

aws iam create-group --group-name DevTeam

Add user to group

aws iam add-user-to-group --user-name DevOps_Intern --group-name DevTeam

Attach custom policy

aws iam put-user-policy --user-name DevOps_Intern --policy-name EC2_T2Micro_Restrict --policy-document file://policy.json

Step 1: Create IAM User

```
aws iam create-user --user-name ec2limiteduser
```

(Verifies user creation)

Step 2: Create IAM Group

```
aws iam create-group --group-name ec2limitedgroup
```

Step 3: Add User to Group

```
aws iam add-user-to-group --user-name ec2limiteduser --group-name ec2limitedgroup
```

Step 4: Create Custom Policy

Suppose your policy JSON is saved locally as `ec2-limited-policy.json`.

Example to create policy:

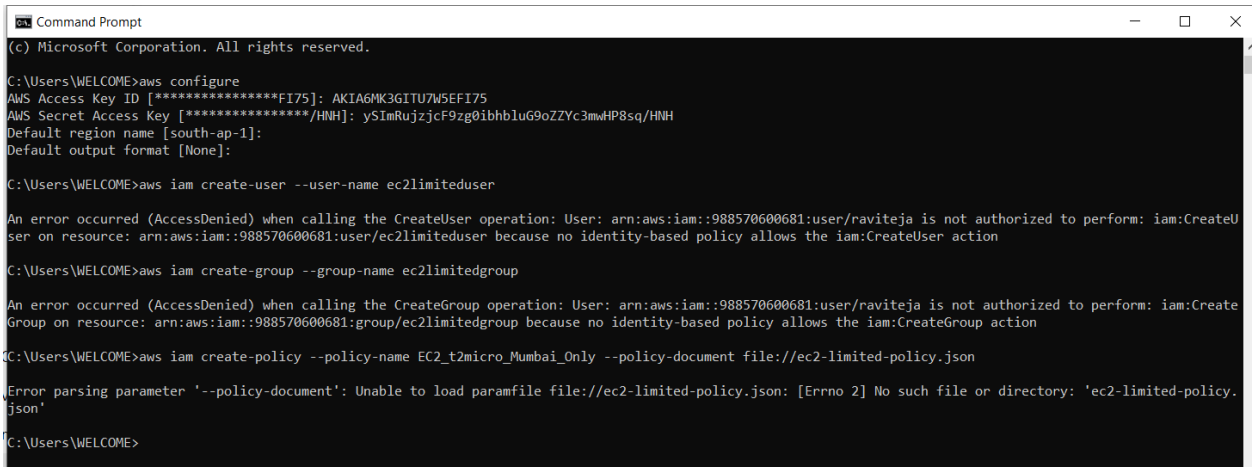
```
aws iam create-policy --policy-name EC2_t2micro_Mumbai_Only --policy-document file://ec2-limited-policy.json
```

This will output the Policy ARN, e.g.:

```
"Arn": "arn:aws:iam::123456789012:policy/EC2_t2micro_Mumbai_Only"
```

Step 5: Attach Policy to Group

```
aws iam attach-group-policy --group-name ec2limitedgroup --policy-arn arn:aws:iam::123456789012:polic
```



```
Command Prompt
(c) Microsoft Corporation. All rights reserved.

C:\Users\WELCOME>aws configure
AWS Access Key ID [****************FI75]: AKIA6MK3GITU7W5EFI75
AWS Secret Access Key [****************/HNH]: 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
Default region name [south-ap-1]:
Default output format [None]:

C:\Users\WELCOME>aws iam create-user --user-name ec2limiteduser

An error occurred (AccessDenied) when calling the CreateUser operation: User: arn:aws:iam::988570600681:user/raviteja is not authorized to perform: iam:CreateU
ser on resource: arn:aws:iam::988570600681:user/ec2limiteduser because no identity-based policy allows the iam:CreateUser action

C:\Users\WELCOME>aws iam create-group --group-name ec2limitedgroup

An error occurred (AccessDenied) when calling the CreateGroup operation: User: arn:aws:iam::988570600681:user/raviteja is not authorized to perform: iam:Create
Group on resource: arn:aws:iam::988570600681:group/ec2limitedgroup because no identity-based policy allows the iam:CreateGroup action

C:\Users\WELCOME>aws iam create-policy --policy-name EC2_t2micro_Mumbai_Only --policy-document file://ec2-limited-policy.json

Error parsing parameter '--policy-document': Unable to load paramfile file://ec2-limited-policy.json: [Errno 2] No such file or directory: 'ec2-limited-policy.
json'

C:\Users\WELCOME>
```

1.Time-Based Access Policies

Theory

AWS IAM lets you use aws:CurrentTime to restrict permissions to specific time windows. Format:
YYYY-MM-DDThh:mm:ssZ (UTC Timezone)

You can combine:

- DateGreaterThan – Access allowed *after* specific time
- DateLessThan – Access allowed *before* specific time

Why Time Restrictions?

Enforce **working hours** only (e.g., 9 AM–6 PM)
Block risky operations after-hours
Support regional compliance rules

Practical Example: EC2 Deployment 9 AM–6 PM IST Only

```
"Condition": {
  "DateGreaterThan": { "aws:CurrentTime": "2024-01-01T03:30:00Z" },
  "DateLessThan": { "aws:CurrentTime": "2024-01-01T12:30:00Z" }
}
```

03:30 to 12:30 UTC = 9:00 AM to 6:00 PM IST

Limitations:

Based on UTC time
Cannot handle recurring daily schedules directly (requires creative use)

Real-Life Example:
A contractor is only allowed to perform database changes during maintenance window, e.g., 11 PM–12 AM UTC.

2.Permission Boundaries

Theory

- Boundaries define **maximum permissions**, even if policies grant more
- Prevents users from escalating privileges
- Boundary policies must be **attached explicitly**

Example Scenario

User Policy: Allows EC2 & IAM
Permission Boundary: Only allows EC2
Boundary Policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:*",
      "Resource": "*"
    }
  ]
}
```

Even if user policy allows iam:CreateUser, boundary blocks it

---

Best Practice:

Use boundaries for:

- Contractors
- Junior developers
- Temporary access restrictions

Real-Life Example:

Dev team can launch EC2, but can't manage other users or modify security groups beyond limit.

3.Policy Writing Practice

Theory

Policies follow JSON structure:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow" or "Deny",
      "Action": [ "service:action" ],
      "Resource": "*"
    }
  ]
}
```

Examples

S3 Read-Only Policy

```
{
  "Effect": "Allow",
  "Action": [ "s3:GetObject", "s3:ListBucket" ],
  "Resource": "*"
}
```

Common for auditors or report download access

Deny RDS Deletion

```
{
  "Effect": "Deny",
  "Action": "rds:DeleteDBInstance",
  "Resource": "*"
}
```

Ensures no accidental DB deletion, even for admins

Tools for Practice:

- **AWS Policy Generator:** https://awspolicygen.s3.amazonaws.com/policygen.html
- **JSONLint Validator:** https://jsonlint.com/

---

4. CLI Simulation (Local Sandbox)

Theory

- AWS CLI can run **dry-run** to test permissions without execution
- Helps practice syntax and output understanding

Examples

Create User Dry Run

aws iam create-user --user-name testuser --dry-run

Returns:
An error occurred (DryRunOperation) when calling the CreateUser operation: Request would have succeeded, but DryRun flag is set.

Check Help Menu

aws ec2 run-instances help
Shows parameters, options, examples
Real-Life Benefit:
Prepare CLI scripts confidently without accidental resource creation or deletion

---

5.Real-World Policy Analysis

Theory

- AWS Managed Policies = Predefined by AWS
- You analyze allowed/denied actions to match job roles

Example 1: AmazonS3ReadOnlyAccess

Allows:
s3:GetObject
s3:ListBucket

Blocks:
Put/Delete actions
Best for:
Auditors, Read-only application users

Example 2: PowerUserAccess

Allows: Most AWS services management
Blocks:IAM user/group/policy management
Best for:
Developers, System Admins (without full root privileges)

Practical Usage:

- Prevent accidental privilege escalation
- Align least privilege principle
- Reduce attack surface